LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 04

(SINGLY LINKED LIST)



Disusun oleh:

MANGGALA PATRA RADITYA

109082500179

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL 04

1. **Soal 1**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string nama;
    Node* next;
};

Node* head = nullptr;

void insertLast(string nama) {
    Node* baru = new Node;
    baru->nama = nama;
    baru->next = nullptr;

    if (head == nullptr) {
        head = baru;
    } else {
        Node* bantu = head;
        while (bantu->next != nullptr) {
            bantu = bantu->next;
        }
        bantu->next = baru;
    }
}

void tampilkanAntrean() {
    cout << "Antrean: ";

    if (head == nullptr) {
        cout << "Kosong";
    } else {
        Node* bantu = head;
        while (bantu != nullptr) {
            cout << bantu->nama;
            if (bantu->next != nullptr) {
                cout << " -> ";
            }
            bantu = bantu->next;
        }
    }
```

```
    cout << endl;
}

void deleteFirst() {
    if (head == nullptr) {
        cout << "Antrean kosong, tidak ada nasabah yang
dilayani."
             << endl;
        return;
    }

    Node* hapus = head;
    cout << "Melayani: " << hapus->nama << endl;
    head = head->next;
    delete hapus;

    tampilkanAntrean();
}

void hapusSemua() {
    while (head != nullptr) {
        Node* hapus = head;
        head = head->next;
        delete hapus;
    }
}

int main() {
    int pilihan;
    string nama;

    do {
        cout << "\n=== SISTEM ANTREAN NASABAH BANK ===" <<
endl;
        cout << "1. Tambah nasabah" << endl;
        cout << "2. Layani nasabah" << endl;
        cout << "3. Keluar" << endl;
        cout << "Pilih menu: ";
        cin >> pilihan;
        cin.ignore(10000, '\n');

        switch (pilihan) {
            case 1:
                cout << "Masukkan nama nasabah: ";
                getline(cin, nama);

                if (nama.empty()) {
```

```
                        cout << "Nama tidak boleh kosong." <<
endl;
                        break;
                    }

                    insertLast(nama);
                    tampilkanAntrean();
                    break;

                case 2:
                    deleteFirst();
                    break;

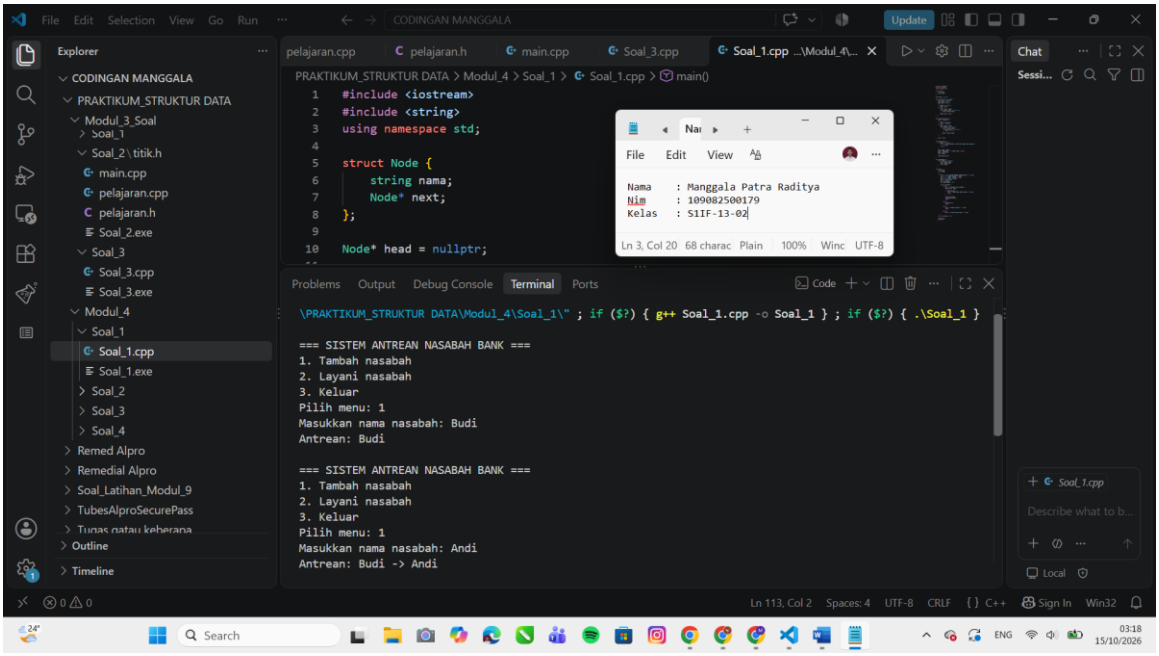
                case 3:
                    cout << "Program selesai." << endl;
                    break;

                default:
                    cout << "Pilihan tidak valid." << endl;
            }

        } while (pilihan != 3);

        hapusSemua();
        return 0;
    }
```

Screenshoot program



Deskripsi program

Pada program ini, saya membuat sistem antrean nasabah bank menggunakan struktur data *Singly Linked List* dengan bahasa pemrograman C++. Program ini digunakan untuk mengatur antrean nasabah yang akan dilayani oleh petugas bank. Saya menggunakan tiga pilihan menu, yaitu menambah nasabah, melayani nasabah, dan keluar dari program.

Pada menu 1, saya memasukkan nama nasabah yang ingin ditambahkan ke dalam antrean. Nasabah baru akan ditempatkan di bagian paling belakang menggunakan fungsi insertLast(). Setelah itu, program menampilkan kondisi antrean terbaru. Pada menu 2, program akan melayani nasabah yang berada di urutan paling depan menggunakan fungsi deleteFirst(). Nama nasabah yang sedang dilayani akan ditampilkan, kemudian nasabah tersebut dihapus dari antrean dan program menampilkan kembali antrean yang tersisa.

Program ini terus berjalan selama pengguna belum memilih menu 3 untuk keluar. Saya juga menambahkan kondisi untuk menangani antrean yang kosong agar program tidak menghapus data yang tidak tersedia. Dari program ini, saya memahami bahwa struktur data *Singly Linked List* dapat digunakan untuk mengelola antrean dengan prinsip FIFO (*First In, First Out*), yaitu nasabah yang masuk terlebih dahulu akan dilayani terlebih dahulu.

2. **Soal 2**

Source Code

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* head = nullptr;

void insertLast(int nilai) {
    Node* baru = new Node;
    baru->data = nilai;
    baru->next = nullptr;

    if (head == nullptr) {
        head = baru;
    } else {
        Node* bantu = head;
        while (bantu->next != nullptr) {
            bantu = bantu->next;
        }
        bantu->next = baru;
    }
}

void insertAfter(int dicari, int nilaiBaru) {
    Node* bantu = head;

    while (bantu != nullptr && bantu->data != dicari) {
        bantu = bantu->next;
    }

    if (bantu != nullptr) {
        Node* baru = new Node;
        baru->data = nilaiBaru;
        baru->next = bantu->next;
        bantu->next = baru;
    }
}

void tampilkanList() {
    Node* bantu = head;

    while (bantu != nullptr) {
        cout << bantu->data;
```

```
            if (bantu->next != nullptr) {
                cout << " -> ";
            }
            bantu = bantu->next;
        }

        cout << endl;
    }

    void hapusSemua() {
        while (head != nullptr) {
            Node* hapus = head;
            head = head->next;
            delete hapus;
        }
    }

    int main() {
        int N, nilai, dicari, nilaiBaru;

        cin >> N;

        for (int i = 0; i < N; i++) {
            cin >> nilai;
            insertLast(nilai);
        }

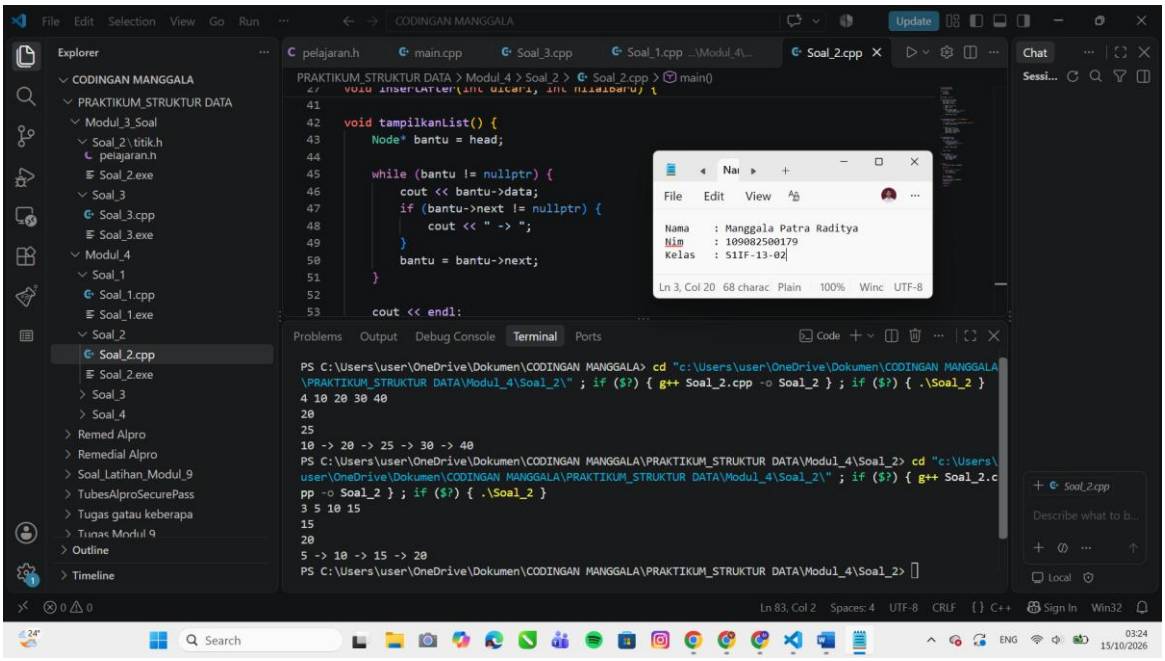
        cin >> dicari;
        cin >> nilaiBaru;

        insertAfter(dicari, nilaiBaru);
        tampilkanList();

        hapusSemua();

        return 0;
    }
```

Screenshoot program



Deskripsi program

Pada program ini, saya membuat program menggunakan bahasa C++ dengan menerapkan struktur data *Singly Linked List*. Program ini digunakan untuk mencari sebuah nilai dalam list, kemudian menyisipkan nilai baru tepat setelah nilai yang ditemukan menggunakan fungsi insertAfter().

Pertama, saya memasukkan jumlah elemen awal beserta nilai-nilai yang akan disimpan ke dalam list. Setiap nilai dimasukkan menggunakan fungsi insertLast() sehingga data tersusun sesuai urutan input. Setelah itu, saya memasukkan nilai yang ingin dicari dan nilai baru yang akan disisipkan.

Program kemudian mencari nilai tersebut dari awal list. Jika nilai ditemukan, program akan membuat node baru dan menyisipkannya tepat setelah node yang memiliki nilai tersebut. Jika nilai tidak ditemukan, isi list tidak mengalami perubahan. Setelah proses selesai, program menampilkan seluruh elemen list dengan tanda panah sebagai pemisah antar data.

Melalui program ini, saya memahami cara melakukan pencarian data dan penyisipan node pada struktur data *Singly Linked List*. Saya juga mempelajari bahwa proses penyisipan dapat dilakukan dengan mengatur pointer pada node yang sudah ada tanpa harus memindahkan seluruh elemen list.

Soal 3

Source Code

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* head = nullptr;

void insertLast(int nilai) {
    Node* baru = new Node;
    baru->data = nilai;
    baru->next = nullptr;

    if (head == nullptr) {
        head = baru;
    } else {
        Node* bantu = head;

        while (bantu->next != nullptr) {
            bantu = bantu->next;
        }

        bantu->next = baru;
    }
}

int hitungJumlah() {
    int total = 0;
    Node* bantu = head;

    while (bantu != nullptr) {
        total += bantu->data;
        bantu = bantu->next;
    }

    return total;
}

void hapusSemua() {
    while (head != nullptr) {
        Node* hapus = head;
        head = head->next;
        delete hapus;
```

```
        }
    }

    int main() {
        int N, nilai;

        cin >> N;

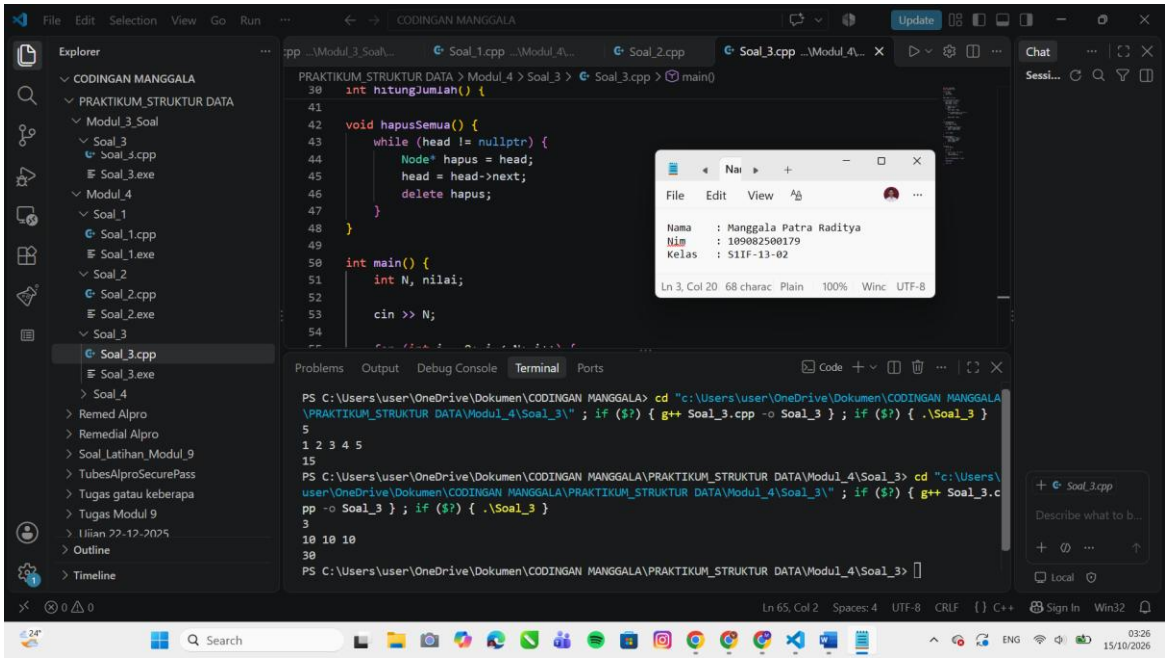
        for (int i = 0; i < N; i++) {
            cin >> nilai;
            insertLast(nilai);
        }

        cout << hitungJumlah() << endl;

        hapusSemua();

        return 0;
    }
```

Screenshoot program



Deskripsi program

Pada program ini, saya membuat program menggunakan bahasa C++ dengan menerapkan struktur data *Singly Linked List* untuk menghitung jumlah seluruh elemen yang tersimpan di dalam list. Program ini memiliki prosedur untuk menambahkan data ke dalam list dan fungsi untuk menghitung total seluruh nilainya.

Pertama, saya memasukkan jumlah elemen yang ingin disimpan, kemudian memasukkan nilai setiap elemen. Data tersebut dimasukkan ke dalam list menggunakan fungsi insertLast(), sehingga setiap elemen tersimpan secara berurutan sesuai dengan urutan input.

Setelah semua data dimasukkan, program menjalankan fungsi hitungJumlah() untuk menjumlahkan seluruh elemen yang ada di dalam list. Proses dilakukan dengan menelusuri setiap node mulai dari head sampai node terakhir. Nilai dari setiap node ditambahkan ke variabel total, kemudian hasil akhirnya ditampilkan ke layar.

Setelah hasil penjumlahan ditampilkan, program menjalankan fungsi hapusSemua() untuk menghapus seluruh node dan membebaskan memori yang digunakan. Melalui program ini, saya memahami cara menelusuri elemen pada *Singly Linked List*, melakukan operasi penjumlahan, serta mengelola memori yang digunakan oleh setiap node.

Soal 4

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string judul;
    int tahun;
    Node* next;
};

Node* head = nullptr;

void insertLast(string judul, int tahun) {
    Node* baru = new Node;
    baru->judul = judul;
    baru->tahun = tahun;
    baru->next = nullptr;

    if (head == nullptr) {
        head = baru;
    } else {
        Node* bantu = head;

        while (bantu->next != nullptr) {
            bantu = bantu->next;
        }

        bantu->next = baru;
```

```
    }
}

void deleteLast() {
    if (head == nullptr) {
        return;
    }

    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;
    }

    Node* bantu = head;

    while (bantu->next->next != nullptr) {
        bantu = bantu->next;
    }

    delete bantu->next;
    bantu->next = nullptr;
}

void tampilkanBuku() {
    cout << "Daftar Buku:" << endl;

    Node* bantu = head;

    while (bantu != nullptr) {
        cout << bantu->judul << " (" << bantu->tahun << ")"
<< endl;
        bantu = bantu->next;
    }
}

void hapusSemua() {
    while (head != nullptr) {
        Node* hapus = head;
        head = head->next;
        delete hapus;
    }
}

int main() {
    int N, tahun;
    string judul;
```

```
        cin >> N;
        cin.ignore();

        for (int i = 0; i < N; i++) {
            cin >> ws;
            getline(cin, judul, ' ');
            string sisa;

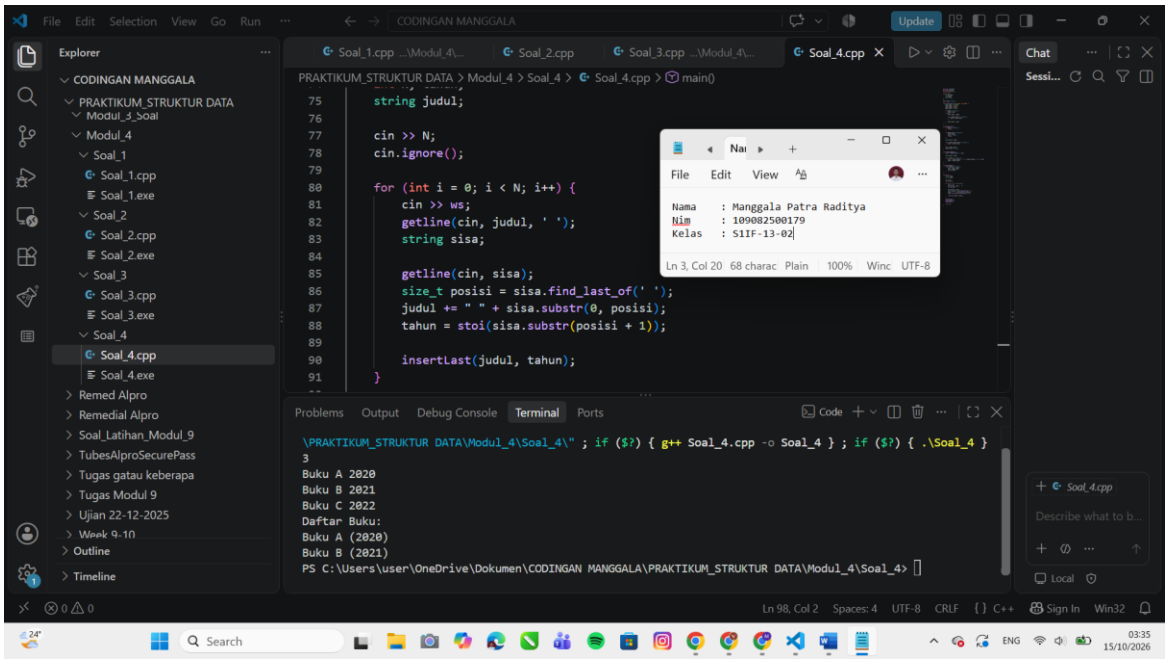
            getline(cin, sisa);
            size_t posisi = sisa.find_last_of(' ');
            judul += " " + sisa.substr(0, posisi);
            tahun = stoi(sisa.substr(posisi + 1));

            insertLast(judul, tahun);
        }

        deleteLast();
        tampilkanBuku();
        hapusSemua();

        return 0;
    }
```

Screenshoot program



Deskripsi program

Pada program ini, saya membuat program menggunakan bahasa C++ dengan menerapkan struktur data *Singly Linked List* untuk mengelola data buku di perpustakaan.

Setiap node menyimpan informasi berupa judul buku dan tahun terbit. Program ini memiliki fitur untuk menambahkan data buku ke dalam list dan menghapus buku yang berada di urutan terakhir menggunakan fungsi deleteLast().

Pertama, saya memasukkan jumlah buku yang ingin disimpan, kemudian memasukkan judul dan tahun terbit setiap buku. Data tersebut disimpan menggunakan fungsi insertLast() agar urutan buku tetap sesuai dengan urutan input. Setelah semua data buku dimasukkan, program menjalankan fungsi deleteLast() untuk menghapus buku yang berada di posisi terakhir.

Proses penghapusan dilakukan dengan menelusuri list hingga menemukan node sebelum node terakhir. Selanjutnya, node terakhir dihapus dan pointer diatur kembali agar list tetap terhubung dengan benar. Setelah itu, fungsi tampilkanBuku() menampilkan daftar buku yang masih tersisa dengan format judul dan tahun terbit. Sebagai contoh, jika saya memasukkan Buku A (2020), Buku B (2021), dan Buku C (2022), maka Buku C akan dihapus sehingga hanya Buku A dan Buku B yang ditampilkan.

Terakhir, saya menggunakan fungsi hapusSemua() untuk menghapus node yang tersisa dan membebaskan memori sebelum program berakhir. Melalui program ini, saya memahami cara mengelola data menggunakan *Singly Linked List*, khususnya dalam melakukan operasi penghapusan data terakhir atau *deleteLast*.